



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS expands the node with the lowest path cost so far, $g(n)$, so it explores outward uniformly in every direction without any sense of where the goal is. A* instead expands the node with the lowest $f(n) = g(n) + h(n)$, where $h(n)$ estimates the remaining cost to the goal. This goal-directed estimate biases the search toward the goal, letting A* skip over large portions of the search space that UCS would still explore, while still guaranteeing an optimal solution when the heuristic is admissible.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

An admissible heuristic never overestimates the true remaining cost. On a 4-way grid, closing a gap of $|x_1 - x_2|$ columns and $|y_1 - y_2|$ rows requires at least that many moves in total, since each move changes only one coordinate by 1 — there is no way to cover the distance in fewer steps. Manhattan distance therefore always equals or underestimates the true shortest-path cost, satisfying admissibility. If the heuristic were not admissible (it sometimes overestimated the true cost), A* could rank a longer, suboptimal path above a shorter one that simply hasn't accumulated enough $g(n)$ yet, and could pop and return that suboptimal path as soon as it reaches the goal. The algorithm

would still terminate and produce a valid path, but it would lose its guarantee of finding the optimal (shortest) one.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No. With diagonal moves allowed, a single step can reduce both the x-gap and the y-gap simultaneously, so the true shortest path can require fewer moves than $|x_1 - x_2| + |y_1 - y_2|$. In that case Manhattan distance overestimates the real cost for diagonal-heavy paths, which violates admissibility. The appropriate replacement is Chebyshev distance, $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$, which exactly matches the minimum number of moves when diagonal and orthogonal steps cost the same. If diagonal moves cost $\sqrt{2}$ instead of 1, straight-line Euclidean distance becomes the correct admissible heuristic instead.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Using distance to only the single nearest food ignores the cost of everything still left to collect afterward, which can lead A* into locally greedy routes that are globally inefficient, such as bouncing between two distant food clusters. A stronger, still-admissible option is a Minimum Spanning Tree (MST) heuristic: build an MST connecting the agent's current position and every remaining food item, using Manhattan distance as edge weights, and use the MST's total weight as $h(n)$. Because any route that visits all the food must cost at least as much as connecting those points together, the MST weight is a valid lower bound and preserves optimality, while giving A* a far more informed estimate than nearest-food distance alone. A cheaper approximation that captures some of the same signal without recomputing an MST every step is nearest-food distance plus the farthest pairwise distance between remaining food items, which accounts for both how far the agent must travel to start and how spread out the remaining food is

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING